

Homework 6: Markov chain Monte Carlo

Bernhard Buss Dario Franzen Aurimas Greicius
Timothy Lauren Nijhuis Carlo Rossetto Yuliyan Slavchev
Görkem Kadir Solun

```
set.seed(42)
```

Problem 15: Markov chain Monte Carlo estimation of the mean

Part a: Proof that $\bar{Y}$ is an unbiased estimator of the mean

Given that

$$\begin{aligned}\bar{Y} &= \frac{1}{N} \sum_{i=1}^N Y_n \\ E[\bar{Y}] &= E\left[\frac{1}{N} \sum_{n=1}^N Y_n\right] = \frac{1}{N} \sum_{n=1}^N E[Y_n] = \\ &= \frac{1}{N} \sum_{n=1}^N \mu = \frac{1}{N} N \mu = \mu\end{aligned}$$

Part b: Approximation of the variance of $\bar{Y}$

Knowing that

$$Var(\bar{Y}) = Var\left(\frac{1}{N} \sum_{n=1}^N Y_n\right) = \frac{Var\left(\sum_{n=1}^N Y_n\right)}{N^2} =$$

When we start from Bienayme's identity an we are multiplying by $\frac{1}{N^2}$:

$$\frac{1}{N^2} Var\left(\sum_{n=1}^N Y_n\right) = \frac{1}{N^2} \sum_{i=1}^N \sum_{j=1}^N Cov(Y_i, Y_j) =$$

$$\frac{1}{N^2} \left(\sum_{i=1}^N \text{Var}(Y_i) + 2 \sum_{1 \leq i < j \leq N} \text{Cov}(Y_i, Y_j) \right) =$$

$$\frac{1}{N^2} \left(N\sigma^2 + 2 \sum_{k=1}^{N-1} (N-k) \text{Cov}(Y_n, Y_{n+k}) \right)$$

Knowing that

$$\rho_k = \frac{\text{Cov}(Y_n, Y_{n+k})}{\sigma^2} \implies \text{Cov}(Y_n, Y_{n+k}) = \sigma^2 \rho_k$$

So

$$\frac{1}{N^2} \left(N\sigma^2 + 2 \sum_{k=1}^{N-1} (N-k) \text{Cov}(Y_n, Y_{n+k}) \right) = \frac{1}{N^2} \left(N\sigma^2 + 2 \sum_{k=1}^{N-1} (N-k) \sigma^2 \rho_k \right) =$$

$$\frac{N\sigma^2}{N^2} \left(1 + \frac{2}{N} \sum_{k=1}^{N-1} (N-k) \rho_k \right) = \frac{\sigma^2}{N} \left(1 + 2 \left(\frac{1}{N} \sum_{k=1}^{N-1} (N-k) \rho_k \right) \right)$$

Since for large N

$$\frac{1}{N} \sum_{k=1}^{N-1} (N-k) \rho_k \approx \sum_{k=1}^{\infty} \rho_k$$

$$\frac{\sigma^2}{N} \left(1 + 2 \left(\frac{1}{N} \sum_{k=1}^{N-1} (N-k) \rho_k \right) \right) = \frac{\sigma^2}{N} \left(1 + 2 \sum_{k=1}^{\infty} \rho_k \right)$$

Since

$$N_{eff} = \frac{N}{1 + 2 \sum_{k=1}^{\infty} \rho_k}$$

$$\sigma^2 \frac{(1 + 2 \sum_{k=1}^{\infty} \rho_k)}{N} = \frac{\sigma^2}{N_{eff}} \approx \text{Var}(\bar{Y})$$

Problem 16: Sampling in the Rain Network

Part a: Joint probability and naive marginalization

$$P(C = T | R = T, S = T, W = T) = \frac{P(C = T, R = T, S = T, W = T)}{P(R = T, S = T, W = T)}$$

The numerator:

$$P(C = T, R = T, S = T, W = T) = P(W = T | S = T, R = T) * P(S = T | C = T) * P(R = T | C = T) * P(C = T) =$$

$$0.99 * 0.10 * 0.80 * 0.50 = 0.0396$$

The denominator:

$$\begin{aligned} P(R = T, S = T, W = T) &= \\ P(W = T|S = T, R = T) * P(S = T|C = T) * P(R = T|C = T) * P(C = T) + \\ P(W = T|S = T, R = T) * P(S = T|C = F) * P(R = T|C = F) * P(C = F) &= \\ 0.99 * 0.10 * 0.80 * 0.50 + 0.99 * 0.50 * 0.20 * 0.50 &= 0.0396 + 0.0495 = 0.0891 \end{aligned}$$

Substituting into the fraction:

$$\begin{aligned} P(C = T|R = T, S = T, W = T) &= \frac{P(C = T, R = T, S = T, W = T)}{P(R = T, S = T, W = T)} = \\ \frac{0.0396}{0.0891} &= 0.\overline{44} = \frac{4}{9} \approx 0.444 \end{aligned}$$

$$P(C = T|R = F, S = T, W = T) = \frac{P(C = T, R = F, S = T, W = T)}{P(R = F, S = T, W = T)}$$

The numerator:

$$\begin{aligned} P(C = T, R = F, S = T, W = T) &= P(W = T|S = T, R = F) * P(S = T|C = T) * P(R = F|C = T) * P(C = T) = \\ 0.90 * 0.10 * (1 - 0.80) * 0.50 &= 0.009 \end{aligned}$$

The denominator:

$$\begin{aligned} P(R = F, S = T, W = T) &= \\ P(W = T|S = T, R = F) * P(S = T|C = T) * P(R = F|C = T) * P(C = T) + \\ P(W = T|S = T, R = F) * P(S = T|C = F) * P(R = F|C = F) * P(C = F) &= \\ 0.90 * 0.10 * (1 - 0.80) * 0.50 + 0.90 * 0.50 * (1 - 0.20) * 0.50 &= 0.009 + 0.180 = 0.189 \end{aligned}$$

Substituting into the fraction:

$$\begin{aligned} P(C = T|R = F, S = T, W = T) &= \frac{P(C = T, R = F, S = T, W = T)}{P(R = F, S = T, W = T)} = \\ \frac{0.009}{0.189} &= 0.047619 \approx 0.048 \end{aligned}$$

$$P(R = T|C = T, S = T, W = T) = \frac{P(R = T, C = T, S = T, W = T)}{P(C = T, S = T, W = T)}$$

The numerator:

$$\begin{aligned} P(R = T, C = T, S = T, W = T) &= P(W = T|S = T, R = T) * P(S = T|C = T) * P(R = T|C = T) * P(C = T) = \\ 0.99 * 0.10 * 0.80 * 0.50 &= 0.0396 \end{aligned}$$

The denominator:

$$\begin{aligned}
P(C = T, S = T, W = T) &= \\
P(W = T|S = T, R = T) * P(S = T|C = T) * P(R = T|C = T) * P(C = T) + \\
P(W = T|S = T, R = F) * P(S = T|C = T) * P(R = F|C = T) * P(C = T) &= \\
0.99 * 0.10 * 0.80 * 0.50 + 0.90 * 0.10 * 0.20 * 0.50 &= 0.0396 + 0.009 = 0.0486
\end{aligned}$$

Substituting into the fraction:

$$\begin{aligned}
P(R = T|C = T, S = T, W = T) &= \frac{P(R = T, C = T, S = T, W = T)}{P(C = T, S = T, W = T)} = \\
\frac{0.0396}{0.0486} &= 0.814815 \approx 0.815 \\
P(R = T|C = F, S = T, W = T) &= \frac{P(R = T, C = F, S = T, W = T)}{P(C = F, S = T, W = T)}
\end{aligned}$$

The numerator:

$$\begin{aligned}
P(R = T, C = F, S = T, W = T) &= P(W = T|S = T, R = T) * P(S = T|C = F) * P(R = T|C = F) * P(C = F) = \\
0.99 * 0.50 * 0.20 * 0.50 &= 0.0495
\end{aligned}$$

The denominator:

$$\begin{aligned}
P(C = F, S = T, W = T) &= \\
P(W = T|S = T, R = T) * P(S = T|C = F) * P(R = T|C = F) * P(C = F) + \\
P(W = T|S = T, R = F) * P(S = T|C = F) * P(R = F|C = F) * P(C = F) &= \\
0.99 * 0.50 * 0.20 * 0.50 + 0.90 * 0.50 * (1 - 0.20) * 0.50 &= 0.0495 + 0.1800 = 0.2295
\end{aligned}$$

Substituting into the fraction:

$$\begin{aligned}
P(R = T|C = F, S = T, W = T) &= \frac{P(R = T, C = F, S = T, W = T)}{P(C = F, S = T, W = T)} = \\
\frac{0.0495}{0.2295} &= 0.215686 \approx 0.216
\end{aligned}$$

In summary:

$$\begin{aligned}
P(C = T|R = T, S = T, W = T) &\approx 0.444 \\
P(C = T|R = F, S = T, W = T) &\approx 0.048 \\
P(R = T|C = T, S = T, W = T) &\approx 0.815 \\
P(R = T|C = F, S = T, W = T) &\approx 0.216
\end{aligned}$$

Part b: Gibbs sampler for Bayesian network implementation

```

tot_samples <- 100

SW_Gibbs_samples <- matrix(0, nrow = 2, ncol = tot_samples)
rownames(SW_Gibbs_samples) <- c("C", "R")

# initialising C and R with random starting states
current_C <- rbinom(1,1,0.5) # rbinom(n=1, size=1, prob) flips a weighted coin
current_R <- rbinom(1,1,0.5)

for (n in seq(tot_samples)) {

  # sample C
  if (current_R == 1) {
    prob_C <- 0.444 # P(C=T | R=T, S=T, W=T)
  } else {
    prob_C <- 0.048 # P(C=T | R=F, S=T, W=T)
  }

  current_C <- rbinom(1, 1, prob_C)

  # sample R
  if (current_C == 1) {
    prob_R <- 0.815 # P(R=T | C=T, S=T, W=T)
  } else {
    prob_R <- 0.216 # P(R=T | C=F, S=T, W=T)
  }
  current_R <- rbinom(1, 1, prob_R)

  # recording the current values
  SW_Gibbs_samples["C", n] <- current_C
  SW_Gibbs_samples["R", n] <- current_R
}

```

Part c: Marginal probability of rain

```

count_rain <- rowSums(SW_Gibbs_samples)["R"]
count_clouds <- rowSums(SW_Gibbs_samples)["C"] # not asked

p_rain <- count_rain / tot_samples
p_clouds <- count_clouds / tot_samples # not asked

```

```
print(p_rain)
```

```
R  
0.37
```

The marginal probability of rain when the sprinkler is on and the grass is wet, $P(R = T | S = T, W = T)$, is 0.37

Part d: Draw 50,000 samples using the Gibbs sampler

```
Gibbs_sampler <- function(tot_samples = 50000) {  
  
  SW_Gibbs_samples <- matrix(0, nrow = 2, ncol = tot_samples)  
  rownames(SW_Gibbs_samples) <- c("C", "R")  
  
  # initialising C and R with random starting states  
  current_C <- rbinom(1,1,0.5) # rbinom(n=1, size=1, prob) flips a weighted coin  
  current_R <- rbinom(1,1,0.5)  
  
  for (n in seq(tot_samples)) {  
  
    # sample C  
    if (current_R == 1) {  
      prob_C <- 0.444 # P(C=T | R=T, S=T, W=T)  
    } else {  
      prob_C <- 0.048 # P(C=T | R=F, S=T, W=T)  
    }  
  
    current_C <- rbinom(1, 1, prob_C)  
  
    # sample R  
    if (current_C == 1) {  
      prob_R <- 0.815 # P(R=T | C=T, S=T, W=T)  
    } else {  
      prob_R <- 0.216 # P(R=T | C=F, S=T, W=T)  
    }  
  
    current_R <- rbinom(1, 1, prob_R)  
  
    # recording the current values
```

```

    SW_Gibbs_samples["C", n] <- current_C
    SW_Gibbs_samples["R", n] <- current_R
  }

  return(SW_Gibbs_samples)
}

```

Part e: Relative frequencies of $R=T$ and $C=T$ and burn-in time

```

tot_samples = 50000

run_1_results <- Gibbs_sampler(tot_samples)
run_2_results <- Gibbs_sampler(tot_samples)

```

```

library(ggplot2)

```

Warning: package 'ggplot2' was built under R version 4.5.2

```

run_1_c_freq <- cumsum(run_1_results["C", ]) / seq_len(tot_samples)
run_1_r_freq <- cumsum(run_1_results["R", ]) / seq_len(tot_samples)
run_2_c_freq <- cumsum(run_2_results["C", ]) / seq_len(tot_samples)
run_2_r_freq <- cumsum(run_2_results["R", ]) / seq_len(tot_samples)

# Combine the data into a single data frame for ggplot
parte_plot_data <- data.frame(
  t = rep(seq(tot_samples), 4),
  Frequency = c(
    run_1_c_freq,
    run_1_r_freq,
    run_2_c_freq,
    run_2_r_freq
  ),
  Variable = rep(c("C = True", "R = True", "C = True", "R = True"), each = tot_samples),
  Run = rep(c("Run 1", "Run 1", "Run 2", "Run 2"), each = tot_samples)
)

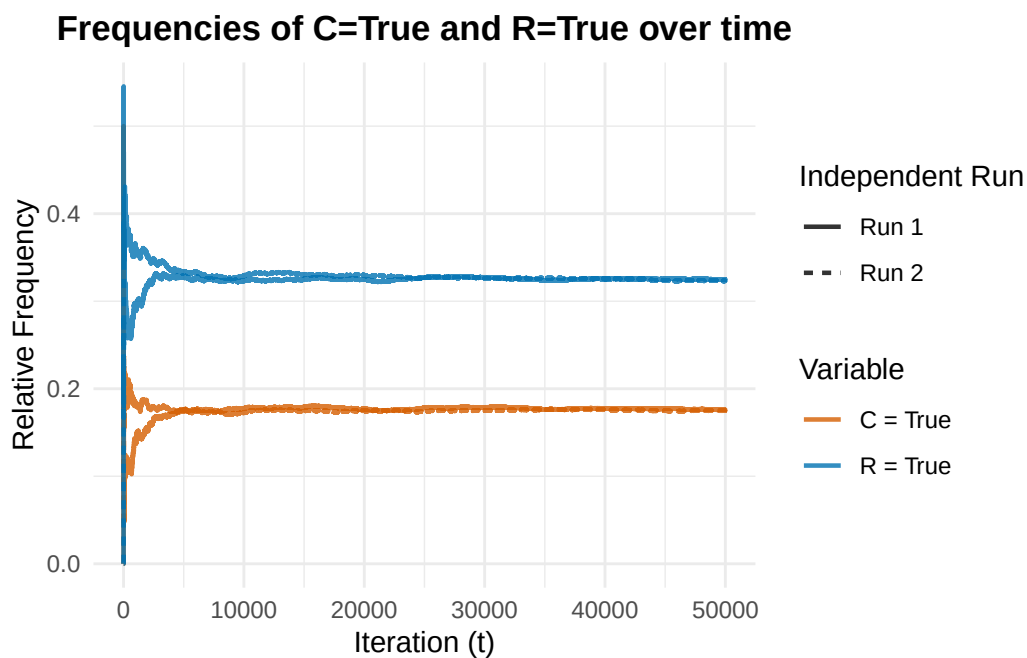
ggplot(parte_plot_data, aes(x = t, y = Frequency, color = Variable, linetype = Run)) +
  geom_line(linewidth = 0.8, alpha = 0.8) +
  theme_minimal() +

```

```

scale_color_manual(values = c("C = True" = "#D55E00", "R = True" = "#0072B2")) +
labs(
  title = "Frequencies of C=True and R=True over time",
  x = "Iteration (t)",
  y = "Relative Frequency",
  color = "Variable",
  linetype = "Independent Run"
) +
theme(
  legend.position = "right",
  plot.title = element_text(hjust = 0.5, face = "bold")
)

```



Based on the trace plot above, a preliminary burn-in of about 20000 iterations seems reasonable.

Part f: Gelman and Rubin test

```

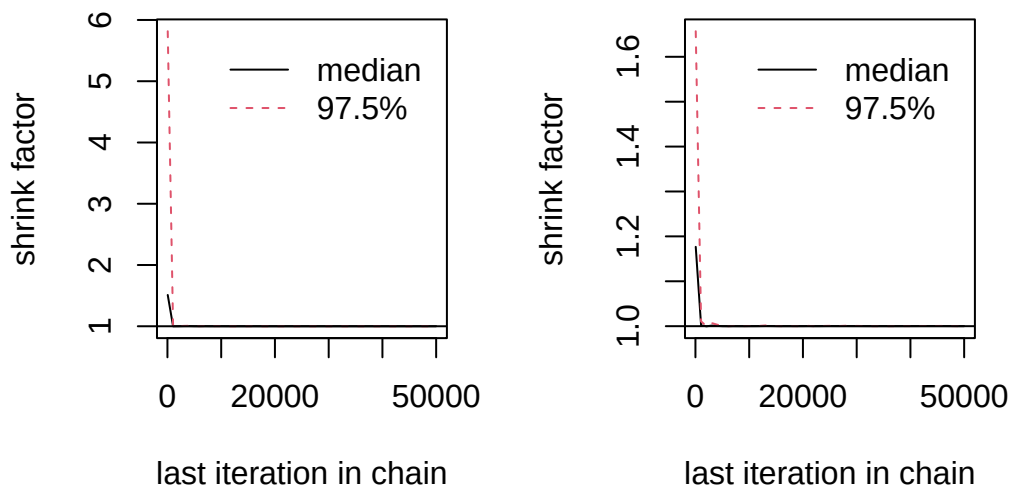
library(coda)

# transpose so iterations are rows and select the raw R and C counts

```

```
combined_mcmc <- mcmc.list(mcmc(t(run_1_results[1:2,])),mcmc(t(run_2_results[1:2,])))
gelman.plot(combined_mcmc, main= "Gelman-Rubin Shrink Factor" )
```

Gelman-Rubin Shrink Factor Gelman-Rubin Shrink Factor



```
gelman.diag(combined_mcmc)
```

Potential scale reduction factors:

	Point est.	Upper C.I.
C	1	1
R	1	1

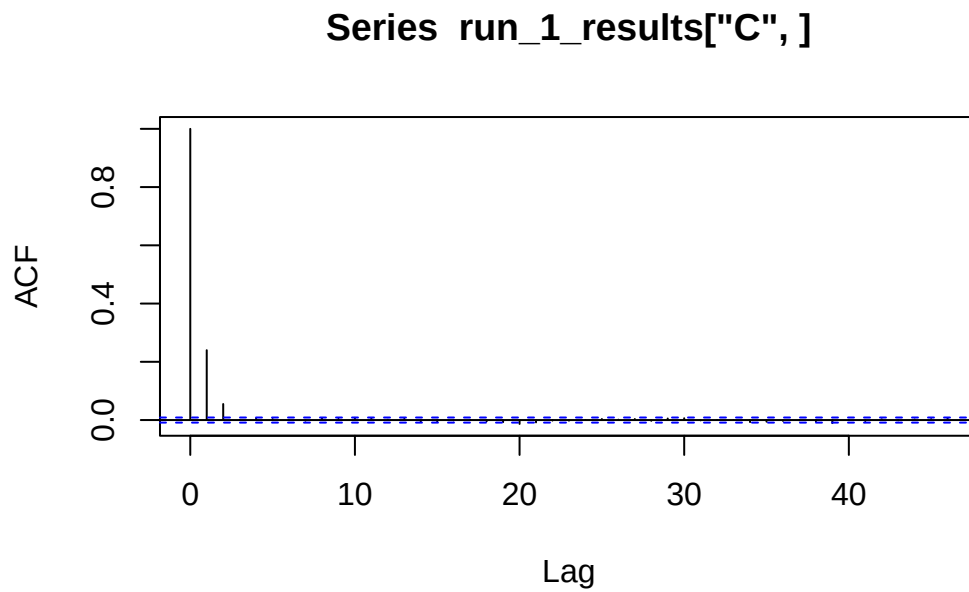
Multivariate psrf

1

For both variables, after about 30000 iterations the shrink factor is very close to 1, so 30000 iterations are recommended for warm-up (burn-in).

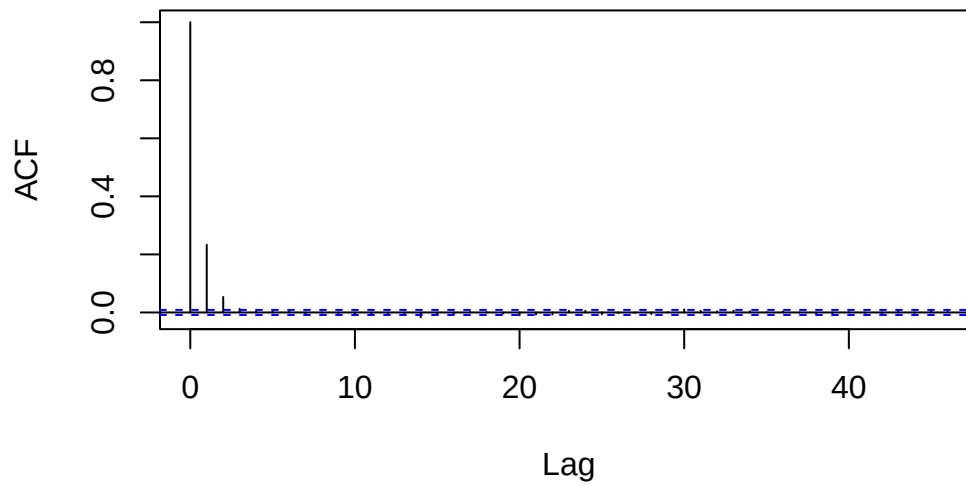
Part g: “Rain and Cloudy” plots

```
acf(run_1_results["C",],type = c("correlation"))
```



```
acf(run_1_results["R",],type = c("correlation")) # lagmax parameter (not included, 30 to 40)
```

Series run_1_results["R",]



After 4 neighbours the autocorrelation function results in a value < 0.05 , therefore the authors would assume that the 5th sample is independent of the first.

Part h: Estimation of $P(R=T|S=T,W=T)$

Burning the first 30,000 samples and considering only every 5th sample (thinning)

```
run_1_raw_burnt <- run_1_results[1:2,30001:50000]
run_2_raw_burnt <- run_2_results[1:2,30001:50000]

run_1_raw_burnt_thinned <- run_1_raw_burnt[,seq(1,ncol(run_1_raw_burnt),5)]
run_2_raw_burnt_thinned <- run_2_raw_burnt[,seq(1,ncol(run_2_raw_burnt),5)]

freq_run_1 <- rowMeans(run_1_raw_burnt_thinned)
freq_run_2 <- rowMeans(run_2_raw_burnt_thinned)

# taking the mean across runs
parth_R_mean <- mean(c(freq_run_1["R"],freq_run_2["R"]))
parth_C_mean <- mean(c(freq_run_1["C"],freq_run_2["C"]))

print(freq_run_1)
```

```

      C      R
0.1725 0.3210

```

```
print(freq_run_2)
```

```

      C      R
0.17300 0.31325

```

```
print(parth_R_mean)
```

```
[1] 0.317125
```

Part i: Analytical computation of $P(R=T|S=T, W=T)$ and comparison with c and h

$$P(R = T|S = T, W = T) = \frac{P(R = T, S = T, W = T)}{P(S = T, W = T)}$$

The numerator:

$$\begin{aligned}
 P(R = T, S = T, W = T) &= P(W = T|S = T, R = T) * P(S = T|C = T) * P(R = T|C = T) * P(C = T) + \\
 &P(W = T|S = T, R = T) * P(S = T|C = F) * P(R = T|C = F) * P(C = F) + \\
 &0.99 * 0.10 * 0.80 * 0.50 + 0.99 * 0.5 * 0.2 * 0.5 = 0.0396 + 0.0495 = 0.0891
 \end{aligned}$$

The denominator:

$$\begin{aligned}
 P(S = T, W = T) &= \\
 &P(W = T|S = T, R = T) * P(S = T|C = T) * P(R = T|C = T) * P(C = T) + \\
 &P(W = T|S = T, R = F) * P(S = T|C = T) * P(R = F|C = T) * P(C = T) + \\
 &P(W = T|S = T, R = T) * P(S = T|C = F) * P(R = T|C = F) * P(C = F) + \\
 &P(W = T|S = T, R = F) * P(S = T|C = F) * P(R = F|C = F) * P(C = F) = \\
 &0.99 * 0.10 * 0.80 * 0.50 + 0.90 * 0.10 * 0.20 * 0.50 + \\
 &0.99 * 0.5 * (1 - 0.8) * 0.5 + 0.9 * 0.5 * (1 - 0.2) * 0.5 = \\
 &0.0396 + 0.009 + 0.0495 + 0.18 = 0.2781
 \end{aligned}$$

Substituting into the fraction:

$$P(R = T|S = T, W = T) = \frac{P(R = T, S = T, W = T)}{P(S = T, W = T)} =$$

$$\frac{0.0891}{0.2781} = 0.320388 \approx 0.320$$

In summary, running two runs with 50000 samples (burning 30,000 and thinning by taking every fifth) resulted in a marginal probability of rain given the grass is wet and the sprinkler is on, $P(R=T|W=T,S=T)$, closer to the analytic truth. In fact, part h (mean of two runs with 50,000 samples) resulted in 0.317125, while part c (with 100 samples) results in 0.37, which is further away from the analytic result of 0.320.

```
comparison_table <- data.frame(
  Method = c(
    "Part c: 100 samples",
    "Part h: 50000 samples with burn-in and thinning",
    "Part i: analytical result"
  ),
  Probability = c(
    p_rain,
    parth_R_mean,
    0.320388
  )
)

knitr::kable(
  comparison_table,
  digits = 3,
  col.names = c("Method", "P(R=T | S=T, W=T)"),
  caption = "Comparison of estimated and analytical probabilities"
)
```

Table 1: Comparison of estimated and analytical probabilities

Method	P(R=T S=T, W=T)
Part c: 100 samples	0.370
Part h: 50000 samples with burn-in and thinning	0.317
Part i: analytical result	0.320